

A Case for Hypergraphs to Model and Map SNNs on Neuromorphic Hardware

Marco Ronzani , Cristina Silvano  *Fellow, IEEE*
DEIB, Politecnico di Milano, Italy

Abstract—Executing Spiking Neural Networks (SNNs) on neuromorphic hardware poses the problem of mapping neurons to cores. SNNs operate by propagating spikes between neurons that form a graph through synapses. Neuromorphic hardware mimics them through a network-on-chip, transmitting spikes, and a mesh of cores, each managing several neurons. Its operational cost is tied to spike movement and active cores. A mapping comprises two tasks: partitioning the SNN’s graph to fit inside cores and placement of each partition on the hardware mesh. Both are NP-hard problems, and as SNNs and hardware scale towards billions of neurons, they become increasingly difficult to tackle effectively. In this work, we propose to raise the abstraction of SNNs from graphs to hypergraphs, redesigning mapping techniques accordingly. The resulting model faithfully captures the replication of spikes inside cores by exposing the notion of hyperedge co-membership between neurons. We further show that the overlap and locality of hyperedges strongly correlate with high-quality mappings, making these properties instrumental in devising mapping algorithms. By exploiting them directly, grouping neurons through shared hyperedges, communication traffic and hardware resource usage can be reduced beyond what just contracting individual connections attains. To substantiate this insight, we consider several partitioning and placement algorithms, some newly devised, others adapted from literature, and compare them over progressively larger and bio-plausible SNNs. Our results show that hypergraph-based techniques can achieve better mappings than the state-of-the-art at several execution time regimes. Based on these observations, we identify a promising selection of algorithms to achieve effective mappings at any scale.

Index Terms—Spiking neural network, neuromorphic computing, HW mapping, hypergraph, partitioning, placement.

I. INTRODUCTION

Our biological brain is remarkably energy efficient in spite of its outstanding capabilities. In light of this observation, **Spiking Neural Networks (SNNs)** have long since been under development to tap into the same event-driven type of intelligence. At the same time, **NeuroMorphic Hardware (NMH)**, inspired by the asynchronous, sparse, and distributed nature of biological neurons, has been built to run SNNs with as little as tens of milliwatts per million neurons [1].

SNNs are composed of neurons connected to one another through synapses. Each neuron integrates over time the spikes – weighted signals – it receives from its synapses and may eventually emit a spike of its own. The network is usually operated in discrete time steps, where all incoming

spikes are propagated and processed, while produced spikes are delivered to their target neurons and queued up for the next time step [2], [3]. In practice, SNNs can be seen as a directed graph, with neurons for nodes and synapses as weighted edges. The present work, in particular, raises such abstraction to that of a **directed hypergraph** by bundling together edges with a common source.

Most NMH is built with a 2D mesh of cores operating in parallel. Each core handles multiple neurons, receiving spikes destined to them, updating their internal state, and emitting the spikes they produce. Cores are interconnected via a network on chip that takes care of spike forwarding. Example implementations include TrueNorth [1], Loihi [4], SpiNNaker [5], and Neurogrid [6]. For a SNN to run on a given NMH, each neuron must be assigned to a specific core by means of a **mapping**. Such a mapping must carefully group connected neurons while respecting hardware limitations, like a maximum of neurons and synapses per core.

The main cost of running a SNN derives from the transmission of spikes between cores [1]. Each hop a spike takes consumes energy and adds latency between successive time steps of the system, that, to retain accuracy, must wait for all spikes to be delivered. Nevertheless, when multiple neurons that are slated to receive the same spike are co-located in the same core, only a single copy of it is sent, relying on replication at the core level to save on communication. Therefore, an optimal mapping is crucial to maximize the efficiency and throughput of the system. A mapping is produced in two steps: **partitioning** of the SNN’s graph, so that each partition can fit onto a virtual NMH core. And **placement**, where each partition is assigned to a specific hardware core [7]. Both of these problems are known to be NP-hard [8]. Thus, with present SNNs exceeding millions of neurons, truly optimal solutions are out of reach. Still, these problems have already been widely studied, under different constraints, for their role in electronic design automation and boolean function manipulation [9], [10], leading to plenty of **heuristics** to handle them effectively. Hence, the expertise needed to handle them has long been established and available.

Several SNN mapping tools, integrating a variety of graph partitioning and placement heuristics, have been developed to seek near-optimal mappings. In TrueNorth’s ecosystem [11], the SNN is assumed to be partitioned at training time, and placement is done sequentially while minimizing the Manhattan distance between connected cores. The Loihi compiler [12] tackles a variant of the problem that includes on-chip SNN training, filling cores successively by mapping nodes in order of connection strength with the present core. Each core’s cluster is seeded by the remaining node

with the most inbound connections. SpiNeMap [13], followed by DFSynthesizer [14], employ the Kernighan-Lin recursive graph partitioning algorithm and a particle-swarm-based placement heuristic. More recently, [7] puts emphasis on scalability and layered SNNs, mapping billions of neurons through simple heuristics. Partitioning proceeds sequentially with the nodes' natural order, saturating each partition before opening the next, while placement initially follows the Hilbert space-filling curve before force-directed refinement. Similarly, EdgeMap [15] sequentially places each node in the partition currently minimizing its cut connections, then relying on a genetic algorithm for placement. Finally, [16] experiments with hierarchical graph partitioning and synapse pruning, while [17] with streaming graph partitioners.

Nonetheless, the problem is far from solved. On one side, many existing techniques have quadratic complexity in the number of SNN neurons or synapses, thus lacking scalability [7]. Others, instead, heavily rely on assumptions about the SNNs' topology to readily leverage the locality in their connections, lacking generalizability to non-layered networks, like liquid state machines [2], [18]. In particular, we find that all existing tools model SNNs as graphs, rather than hypergraphs. This leads them to exploit foremost the direct connections between neurons as a means to decide their proximity in the final mapping. However, that only brings spike sources closer to their destinations, overlooking the origin and benefits of spike replication in NMH; namely, that it happens when multiple common destinations are in the same core. What's more, when different destination neurons are brought together, they can share the hardware resources that handle inbound spikes – something that does not occur between source and destination. We also observe that reliance on individual edges leads some performance models to overestimate spike traffic altogether [7], [15].

For these reasons, we advocate a shift to the hypergraph model: it retains all the expressive power of graphs while explicitly capturing sets of nodes that partake in the same hyperedges and thus receive the same spikes, thereby exposing opportunities for replication and resource sharing.

In this work, we reformulate the problem of mapping spiking neural networks on neuromorphic hardware in terms of hypergraphs, focusing on fully capturing the circulation of spikes within their abstraction. From this formalization, we identify two key properties of a good mapping, core-level synaptic reuse of spikes and locality of connected cores, and trace back their origin to two affinity measures defined between co-located nodes of a hypergraph. Our intuition is that pursuing these properties serves as a guiding principle for effective mapping algorithms, making them not just descriptive of good solutions but prescriptive for their design. Moreover, we empirically verify that good solutions indeed correlate with high exploitation of these two properties.

To act on our case for hypergraphs, we actively leverage them in several mapping algorithms that apply our intuition. Currently, much of the literature on hypergraphs remains to be explored when applied to the SNN-on-NMH mapping problem [9], [19], [20], [21]. Drawing from it, we implement a hierarchical partitioning scheme adapted to the constraints

of NMH and introduce a placement scheme based on a discretization of the hypergraph's spectrum, which naturally closes the gap between highly connected partitions. We also revisit within our framework several partitioning and placement techniques from existing SNNs mapping tools: sequential partitioning, Hilbert curve, force-directed, and minimum distance placement. In addition, we propose a new heuristic for hypergraph partitioning that, driven by the synaptic reuse property, works by bundling together nodes that share membership in many hyperedges. For each algorithm, we study its computational complexity and trade-offs in terms of speed versus quality of results. Finally, we conduct experiments to evaluate all combinations of the presented heuristics and the role hypergraph properties played in each of them. In doing so, and to the best of our knowledge, we are the first to include recurrent and biologically plausible SNNs, alongside classic feedforward ones, in our validation. This enables us to identify the most suitable set of algorithms to handle the mapping problem for current and future large-scale SNNs.

This paper is organized as follows: in Section I, we provide the context and motivation for our work. In Section II, we introduce our notation for SNNs and NMH, while in Section III, we formalize the mapping problem accordingly. With Section IV, we present existing and novel techniques to handle h-graph partitioning and placement. Then, in Section V, we discuss our experimental evaluation and results. Finally, with Section VI, we summarize and comment on our findings.

II. PRELIMINARIES

A. Spiking Neural Networks

Spiking neural networks are a computational paradigm for artificial intelligence inspired by the structure and functions of biological neurons. Unlike Artificial Neural Networks (ANNs), they retain and leverage the asynchronous and distributed calculation properties found in nature. As such, they propagate information through events, represented by spikes, that leave a neuron through its axon to then be received by other neurons connected to it via weighted synapses. Upon receiving a spike, a neuron updates its internal mechanisms and may emit another spike of its own. For simplicity, spike propagation and generation of future spikes are typically performed in discrete time steps [3].

SNNs can be built in several ways. They can be converted from a feedforward ANN, retaining most of the original model's accuracy [22], [23]. Crucially, this results in layered SNNs, with distinct, ordered groups of neurons corresponding to the original network's layers and all synapses concentrated in between those groups [2], [7]. Alternatively, SNNs can be trained from scratch. While methods in this field have yet to produce models with accuracy rivaling ANN on traditional datasets, they consistently find success in event-driven and temporal tasks [3], [24]. In those natively trained SNNs, we may find any arbitrary topology of synapses, consider, for example, liquid state machines, which present cycles in the form of feedback connections [18], [25], [26]. Such bio-like SNNs have been called cyclic, feedback, or recurrent.

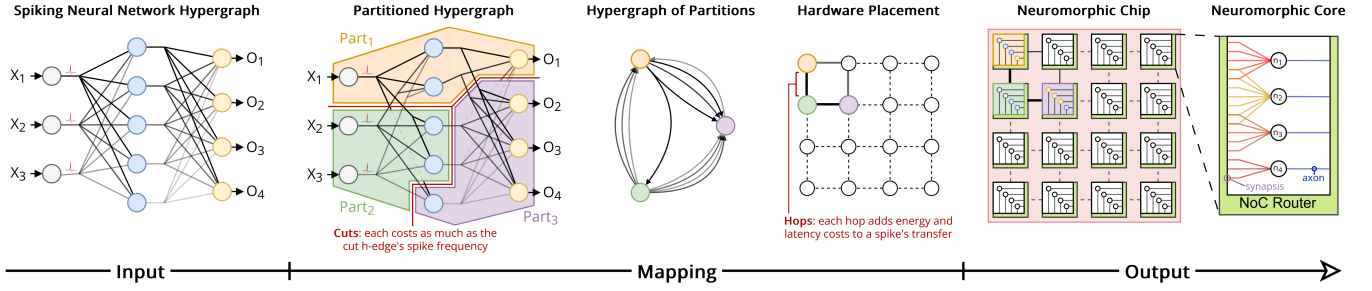


Fig. 1: Mapping steps: from a spiking neural network to its partitioning and placement on neuromorphic hardware.

We represent a SNN as a directed weighted hypergraph (*h-graph*), where each hyperedge (*h-edge*) has a single source. Thus, nodes represent neurons, while *h-edges* the axons departing once from each neuron.

$$\begin{aligned} G_S &:= (N, E_S, w_S), \\ E_S &:= \{e = (s, D) \mid s \in N, D \subseteq N\}, \\ w_S &: E_S \rightarrow \mathbb{R}^+. \end{aligned} \quad (1)$$

Let G_S be the *h-graph*, each $n \in N$ a node and each $(s, D) \in E_S$ one of its *h-edges*. Then, s is the *h-edge's* source while D is the set of its destinations. Lastly, w_S is a function assigning a weight to each *h-edge*, corresponding to its spike frequency in the SNN. From each neuron of a SNN departs a single axon; thus, any $s \in N$ will appear exactly once as a source among *h-edges*. This means there is a one-to-one correspondence between *h-edges* and neurons, and an *h-edge's* spike frequency can also be interpreted as that of its source neuron.

B. Neuromorphic Hardware

Neuromorphic hardware accelerators for SNNs consist of a spatial mesh of cores. A core handles several neurons, storing their parameters, updating them upon spike arrival, and sending out produced spikes. Every core has a router that connects it to the others through a network on chip and takes care of spike multicast and forwarding between different cores. Global synchronization only occurs across the computation's discrete time steps, once all cores have handled the spikes generated by the previous step. Otherwise, to minimize power consumption, NMH mimics SNNs in being mostly asynchronous, with a core activating only upon receiving a spike. Consequently, its major performance bottlenecks are the energy and time spent on spike movements, spike transmission latency in particular determines the global time step duration and thus the system's throughput [1], [2], [12]. To mitigate these costs, NMH cores support spike replication: each core receives at most one instance of any spike and internally dispatches it to as many neurons as needed. This limits the copies of a spike that may need to circulate to one

per core. See Fig. 2 for an example of such multiple local deliveries. Such architecture is typically scalable to multiple chips connected in a higher-order mesh. However, since off-chip communication is less predictable than that on-chip, this work's hardware model focuses on single-chip systems.

Formally, NMH is here modeled as a 2D lattice of cores of known width and height:

$$H := \{h = (x, y) \in \mathbb{N}^2 \mid 0 \leq x < \text{width}, 0 \leq y < \text{height}\}. \quad (2)$$

Each core $h \in H$, represented by its coordinates, can handle a maximum of C_{npc} neurons and C_{apc} incoming axons in common between such neurons. In other terms, when mapping a SNN's *h-graph* over the hardware, at most C_{npc} nodes can share a common core, and their distinct inbound *h-edges* can't be more than C_{apc} . Each source of spikes seen by a core usually needs a dedicated hardware queue, resulting in the above limits on the maximum number of axons that neurons in a core can collectively connect to. Then, depending on the implementation, inside a core, each neuron could be allowed to have a synapse with all such axons, as in TrueNorth [1], or there might be a further limit on the number of synapses per core, $C_{spc} < C_{npc} \cdot C_{apc}$, as in Loihi [4].

III. MAPPING MODEL

Given a SNN G_S , we model one of its possible partitionings through a surjective function $\rho : N \rightarrow P$ that maps each node of the SNN to a partition $p \in P$. By pushing forward G_S through ρ , we define G_P as the new *h-graph* that emerges among the partitions of G_S :

$$\begin{aligned} G_P &:= (P, E_P, w_P), \\ P &:= \{\rho(n) \mid n \in N\}, \\ E_P &:= \{e_P = (\rho(s), \{\rho(d) \mid d \in D\}) \mid e_S = (s, D) \in E_S\}, \\ w_P(e_P) &:= w_S(e_S). \end{aligned} \quad (3)$$

In the so-obtained *h-graph* G_P , each node $p \in P$ is a partition assigned by ρ , hence $|P| \leq |N|$. Then, *h-edges* between partitions are constructed as E_P and, for each *h-edge*, its weight is defined by $w_P : E_P \rightarrow \mathbb{R}^+$. We may subsequently merge *h-edges* with identical source and destinations by adding together their weights. Here, the one-to-one relation between *h-graph* nodes and *h-edges* is lost, as multiple *h-edges* may depart from the same partition. A partitioning is valid iff it fits the hardware's constraints. The preimage size of each codomain element of ρ , being the size of each partition, is limited by C_{npc} :

$$\forall p \in P \mid |\{n \in N \mid \rho(n) = p\}| \leq C_{npc}. \quad (4)$$

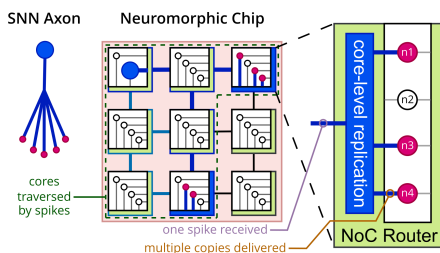


Fig. 2: Example of axon mapping and core-level spike replication.

The count of distinct h-edges inbound to the same partition is limited by C_{apc} :

$$\forall p \in P \quad |\{(s, D) \in E_S \mid \exists n \in D, \rho(n) = p\}| \leq C_{apc}. \quad (5)$$

The overall count of inbound connections to the same partition is bounded by C_{spc} :

$$\forall p \in P \quad \sum_{(s, D) \in E_S} |\{d \in D \mid \rho(d) = p\}| \leq C_{spc}. \quad (6)$$

The cost function to minimize in an optimal partitioning is the *weighted connectivity*, or $\lambda - 1$ metric, between partitions:

$$Conn(G_P) = \sum_{e_P=(s,D) \in E_P} w_P(e_P) \cdot |D|. \quad (7)$$

This equates to minimizing the number of partitions each h-edge connects, by paying once the h-edge's weight per connected partition [19]. Incidentally, this is the same as minimizing the average amount of spikes that need to transit between cores once the SNN is mapped on NHW [7].

Finally, a placement (layout) is modeled by another injective function $\gamma : P \rightarrow H$, assigning (placing) each partition to a hardware core. Fig. 1 summarizes all discussed steps. The post-layout performance metrics minimized by an optimal mapping are listed in Table I. They are an adaptation to hypergraphs of the formulae in [7] and include the system's energy consumption, latency, and interconnect congestion. Their estimation is conditioned on low-level measurements of the hardware's routers and wires that build up the cost of carrying spikes between cores. We take the measurements used in our experiments, as well as the hardware constraints, from Intel's Loihi [4] ("small") and [7] ("large"), see Table II.

The present mapping model ensures that spike replication capabilities of NMH cores are accounted for. This occurs when multiple neurons inside the same partition (and eventually, core) share an h-edge. In such a case, the hardware only transmits the h-edge's spikes once towards the core, relying

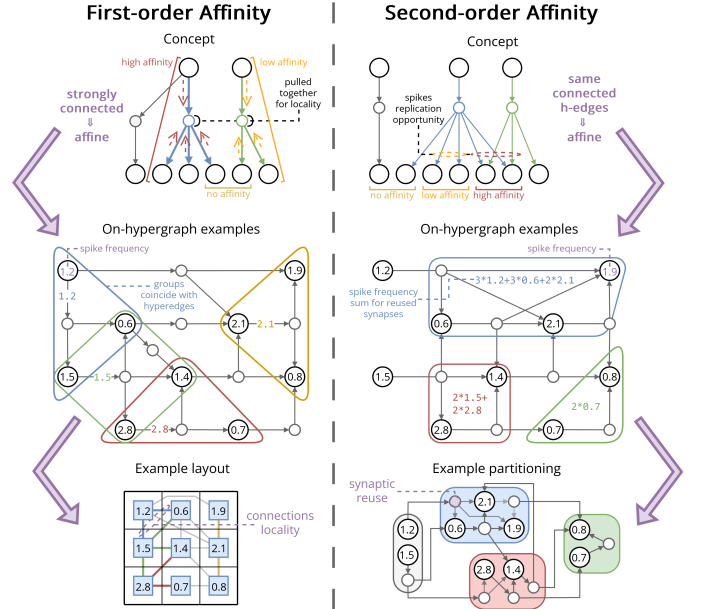


Fig. 3: Illustration and examples of first- and second-order affinity.

on it to, very cheaply, integrate the spike on each neuron needing it. For instance, Eq. 7 accounts for this by paying an h-edge's weight in connectivity only once per partition. Note that the cost of input spikes has not been modeled, as those only arrive once in hundreds or thousands of system steps, thus having a negligible impact on the final metrics [2].

A. Mapping Properties

From the presented formalization and objectives, we identify two properties that characterize good mappings. A mapping's partitioning shall minimize connectivity by maximizing the replication of spikes inside hardware cores, inherently reducing the number of copies of a spike that are transmitted. This manifests as *synapses reusing spikes* – a property we call *synaptic reuse* – when multiple neurons in the same partition, and thus hardware core, receive spikes from a common source (axon). In turn, this implies that the core only allocates physical resources to receive such a source's spikes once, hence being able to fit more nodes. A mapping's layout shall then minimize the above metrics by limiting the overall distance traveled by inter-core spikes, thereby reducing traffic on the interconnect. This leads to the property of *connections locality*, reflecting how closely the destinations of an h-edge are placed – ideally within a small neighborhood of cores. The more confined an h-edge is, the less congestion its spikes cause, leading to more consistent, timely deliveries and reduced latency across global time-steps. This also seamlessly creates an opportunity for the hierarchical multicasting of spikes, on architectures that implement such a feature [4]. Connections locality is the layout counterpart of synaptic reuse, that relies on co-member nodes getting closer, rather than sharing the same core. Ultimately, synaptic reuse and connections locality are complementary, both being fundamental mechanisms to drive mapping algorithms.

At the h-graph level, we thus pinpoint two measures that can effectively guide the construction of mappings. These come in the form of affinities that advocate for certain nodes

Symbol	Definition
$\ \cdot\ $	L1 norm (Manhattan distance)
E_R, L_R	Energy and latency for a spike's routing
E_T, L_T	Energy and latency for a spike's transmission between two cores
$\tau(h, h_s, h_d)$	Probability of a spike being routed through core h when going from core h_s to h_d (see [7] for details)
$Rect(h_1, h_2)$	Set of lattice points contained in the closed square defined by the opposite corners h_1 and h_2
Metric	Expression
Energy consumption	$\sum_{e=(s,D) \in E_P} \sum_{d \in D} w_P(e) (\ \gamma(s) - \gamma(d)\ (E_R + E_T) + E_R)$
Average latency	$\frac{\sum_{e=(s,D) \in E_P} \sum_{d \in D} w_P(e) (\ \gamma(s) - \gamma(d)\ (L_R + L_T) + L_R)}{\sum_{e \in E_P} w_P(e)}$
Average congestion	$\sum_{e=(s,D) \in E_P} \sum_{d \in D} w_P(e) \sum_{h \in Rect(\gamma(s), \gamma(d))} \tau(h, \gamma(s), \gamma(d))$

TABLE I: SNN mapping performance metrics. Adapted from [7].

NMH Cost	Value	Constraint	Value	
			small	large
E_R	1.7 pJ	C_{npc}	1024	4096
L_R	2.1 ns	C_{apc}	4096	65536
E_T	3.5 pJ	C_{spc}	16384	262144
L_T	5.3 ns	<i>width, height</i>	64 × 64	

TABLE II: Reference hardware costs and constraints [4], [7].

to be mapped in close proximity of one another. *First-order affinity*, the weight of the direct connection between all pairs of nodes under the same h-edge. The stronger the connection, the closer such nodes shall be to reduce its cost. *Second-order affinity*, the weight of co-membership, that is, the cumulative weight of the h-edges a set of nodes partakes in together [27]. In other words, grouping nodes with overlapping sets of connections amortizes their h-edges' connections cost over them. Synaptic reuse emerges when high-second-order-affinity nodes share the same partition. Conversely, connections locality is reliant on the relative adjacency of nodes with high first-order affinity. As a result, second-order affinity shall guide the partitioning, while first-order affinity the placement. Fig. 3 provides a visual intuition for this. Finally, the key challenge of heuristics is identifying which affine nodes it is best to map closer together while performing as few iterations as possible over the h-graph.

In summary. To reduce **cuts** during **partitioning**, a mapping exploits spike **replication** in NMH cores through **synaptic reuse** by grouping nodes with high **second-order affinity**. To limit spike **hops** after **layout**, via shorter **Manhattan distances** on NMH, a mapping increases **connections locality** by pulling closer partitions with high **first-order affinity**.

B. Towards Hypergraphs

In graph form, a SNN's source node connects to multiple destinations with a different edge for each, thus losing access to the notions of first- and second-order affinity. Practically, this hinders the quantification and realization of synaptic reuse and connections locality, where you need to reason in terms of overlapping connections and distance between co-member nodes. With h-graphs, instead, h-edges naturally group together nodes under a joint source, exposing these properties. In fact, porting to h-graphs the metrics in Eq. 7 and Tab. I from [7] corrected an overestimation of w_p that did not account for synaptic reuse due to the elicitation of edges, rather than h-edges.

Ultimately, the graph model's lack of a native coalesced representation of destinations with a shared source adds overhead to algorithms that rely on some measure of affinities for guidance. Partitioning struggles to identify nodes with overlapping inbound sets, while placement cannot pull together h-edges as a whole and uniformly close the gap between all their nodes. In turn, this encourages mappings to solely rely on source-destination connections, missing out on reuse and locality. Arguably, building an index that groups edges by source could mitigate the issue. However, we object that doing so already transitions from a graph towards the proposed hypergraph abstraction. In this sense, many approaches already subtly lean towards hypergraphs, but, by not fully adopting them yet, they encounter limitations in problem modeling and algorithm design.

For these reasons, we concluded that hypergraphs are better suited to model SNNs throughout the mapping process. Nonetheless, it is trivial to decompose each h-edge in multiple edges to use a graph representation if needed. As a last note,

the hypergraph formulation more closely mirrors biological neural networks, with each h-edge corresponding to an axon.

IV. PROPOSED ALGORITHMS

Both constrained h-graph partitioning and placement are NP-hard problems; thus, it is computationally unfeasible to aim for an optimal solution when millions of nodes and h-edges are involved. On account of this, SNN mapping techniques shall consist of heuristics that, paired with suitable assumptions, trade off execution time and solution quality [28]. Even more so, heuristics must be scalable, retaining a feasible time-to-solution (e.g. tens of hours) on a single machine while striving for near-optimal mappings, even when growing SNNs to "human-scale" – tens of billions of neurons. As such is the end goal of neuromorphic computing [1].

We will now describe several heuristics. Some are adapted from research on SNN mapping, others are based on hypergraphs or EDA studies that have not yet been applied in this context. Moreover, we highlight how each heuristic makes effective use of the properties defined in Sec. III-A.

Henceforth, we rely on a generic input h-graph to ease the discussion of algorithmic complexity: let n be its number of nodes, e its number of h-edges, and d the average h-edge cardinality. From which, each node is on average touched by $h := e \cdot d / n$ h-edges. Actually, with our definition of SNNs (Sec. II-A) we have $n = e$ and so $d = h$; we will use this fact to standardize all reported complexities. In general, $n, e \gg d, h$, as SNNs tend to favor localized connectivity [3], thus the complexity of examined algorithms will foremost be linear in the h-graph's size, n or e . Consequently, it will be the additional dependencies on d, h , or other constants that make up the difference in final running time.

When outlining algorithms, we adopt the following data structures. Let an h-graph be represented as a list of h-edges, each h-edge being a pair consisting of a list of nodes, whose first element is the source, and a spike-frequency weight. Nodes are identified by consecutive integers. Lastly, two auxiliary indices provide constant-time access to the set of h-edges inbound to a node and to its outbound h-edge [29].

A. HyperGraph Partitioning

Hypergraph partitioning involves separating an h-graph's nodes in disjoint sets while minimizing the number and cardinality of hyperedges that get split among multiple partitions. This problem can come in many forms depending on its constraints and quality metrics. In our case, we have upper limits on nodes and connections per partition, distinct inbound h-edges per partition, and total partitions, but we are allowed to arbitrarily reduce the number of partitions. This "distinct" keyword, especially, is what enables us to fit many nodes per partition, saving cuts by leveraging synaptic reuse. The ideas behind presented algorithms are shown in Fig. 4.

IV-A1. Hierarchical Partitioning: Inspired by hMETIS [30] and KaHiPar [19], [29], this is a multi-level hypergraph partitioning method with refinement based on the Fiduccia-Mattheyses algorithm [8]. Seeing that existing implementations aim to create an exact, given number of balanced

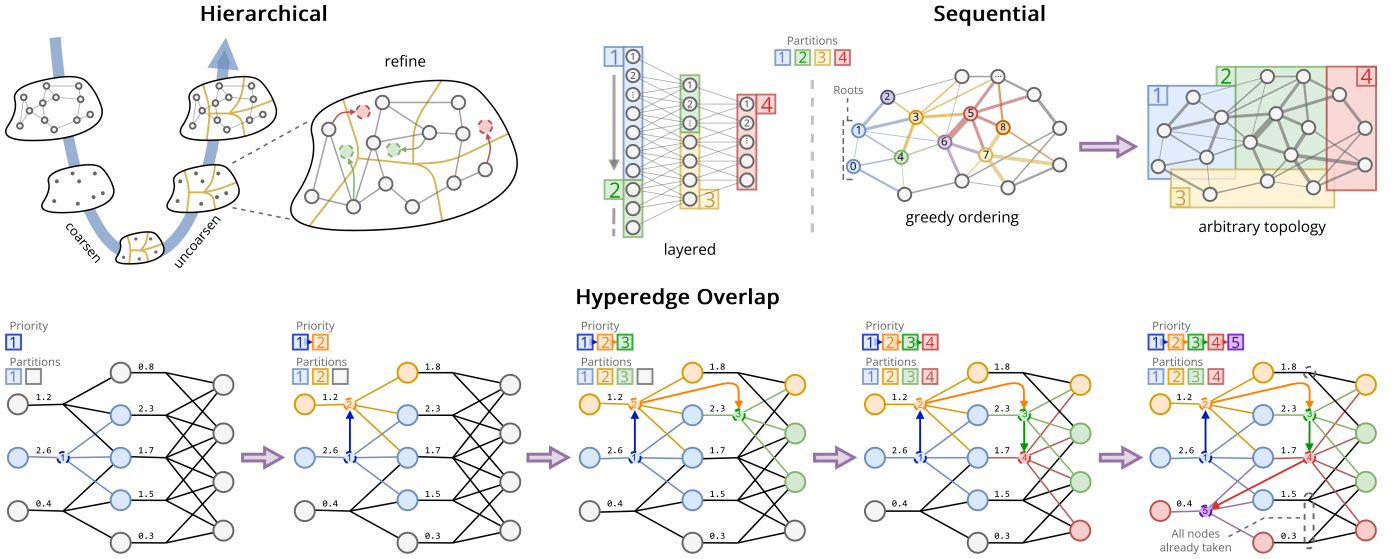


Fig. 4: Overview of the considered partitioning algorithm.

partitions [9], we shall rework them to minimize partitions under the constraints from Sec. II. The method begins by constructing a hierarchy of progressively coarser representations of the input SNN hypergraph through weighted edge-coarsening [10]. This leads to an initial partitioning that is then refined while the hierarchy is undone. Crucially, such a multi-level approach makes the use of iterative refinement techniques, usually reserved for small h-graphs, feasible.

During a coarsening round, nodes are iterated in random order. For each node, all yet unpaired nodes partaking in the same h-edges are considered and scored by the total weight of the h-edges linking them. Essentially, a pair-wise evaluation of second-order affinity. A pair is formed between the present node and, if any, the highest scoring one such that the two together do not violate per-partition constraints. Before the next round, each pair is placed in the same partition and subsequently treated as a single node, while duplicate h-edges are merged. Coarsening stops when no pairs can be formed or there are exactly $\lceil |n|/C_{npc} \rceil$ nodes, providing the initial, almost minimized number of partitions; let it be k .

Each node of the final coarsened graph is a partition of our candidate solution. Such partitioning is then incrementally projected back onto finer levels by replacing each node with the pair that formed it, if any. During each uncoarsening round, local refinement is applied using a gain-based heuristic. The algorithm evaluates whether moving a neuron to a neighboring partition (e.g. one its outbound h-edges connect to) reduces the overall communication cost as defined by Eq. 7. Nodes are visited in random order, and all advantageous moves are greedily applied so long as constraints are respected. Still, the refinement process is efficient, as coarse moves implicitly act on multiple nodes, while cost variations remain computed locally.

The result is an $O(e \cdot d^2 + e \cdot d \cdot k)$ complexity, dominated by coarsening: $O(n \cdot h \cdot d) = O(e \cdot d^2)$. Refinement, instead, runs in $O(n \cdot h \cdot k + e \cdot d) = O(e \cdot d \cdot k)$, achieved by precomputing node counts per partition per h-edge via a single scan of all h-edges, costing $O(e \cdot d)$. The hierarchy spans $O(\log n/k)$ levels, each roughly halving the h-graph’s size, amounting

to a small multiplicative constant to the complexity. Notably, there exist approaches that avoid enumerating all node pairs during coarsening, removing the quadratic dependency on d by approximating pair affinity [19]. However, we opted for the original, exact, edge-coarsening for two reasons: first, our implementation strictly relies on coarsening to form a minimal number of partitions for the given constraints, which is different from standard literature where k is fixed a priori. Second, in this way, our work can effectively depend on hierarchical partitioning as a source of near-optimal solutions for more scalable algorithms to compare against.

IV-A2. Hyperedge Overlap-based Partitioning: This is our novel greedy algorithm that constructs one partition after the other, placing nodes in them while solely focusing on second-order affinity to maximize synaptic reuse. This is achieved through a single sweep of all h-edges, for each laying all its nodes. To address reuse, the order of h-edges is dynamically adjusted such that the next one always has the highest overlapping set of connected nodes with those seen immediately before it. Alg. 1 presents the full approach¹.

First, we sort h-edges in descending order of their number of connections (line 8). This will be the order in which our outer loop sees h-edges, thus prioritizing those that contribute most to connectivity by involving many nodes. We also set up an initial empty partition (line 10), where we will allocate nodes until further doing so would violate constraints, causing the creation of a new partition (line 25). Then, for each h-edge, we go over its destination nodes and assign them to the current partition. This is done by continuously selecting the node whose set of inbound h-edges introduces the fewest new inbound h-edges to the current partition; in case of a tie, the node with the largest inbound set is chosen, thereby maximizing overlap after minimizing additions (line 21). Exceptionally, we first also assign the h-edge’s source node, if it isn’t part of any other h-edge (line 19). Simultaneously, an addressable priority queue is

¹Convention: argmax^* mean any arbitrary maximizer, likewise for argmin^* . Then, $\operatorname{argmin}^{\text{lex}}$ does the same but on tuples ordered lexicographically.

Algorithm 1 Partitioning by Hyperedge Overlap

Input: $G_S = (N, E_S, w_S)$, C_{npc} , C_{apc} , C_{spc} , $|H|$
Output: $\rho : N \rightarrow P$

```

1: for  $n$  in  $N$  do
2:    $\rho(n) = p_{null}$  ▷ blank partitions assignment
3:    $inbound(n) = \{(s, D) \in E_S \mid n \in D\}$  ▷ precompute inbound sets
4:    $outbound(n) = \{(s, D) \in E_S \mid s = n\}$  ▷ outbound singletons
5: for  $e$  in  $E_S$  do
6:    $size(e = (s, D)) = |D| + 1$  ▷ remaining connections count per h-edge
7:    $pq(e) = 0$  ▷ addressable priority queue
8:  $sort(E_S, key : size(\cdot), descending)$ 
9:  $seen = \{\}$  ▷ already visited h-edges
10:  $p_{curr} = next-partition(P)$ 
11:  $npc = 0, spc = 0, apc = \{\}$  ▷ current partition's constrained variables
12: while  $|seen| < |E_S|$  do
13:   if  $\exists a \in E_S$  s.t.  $pq(a) > 0$  then
14:      $e = (s, D) = argmax_{b \in E_S \setminus seen} w_S(b) \cdot pq(b)$  ▷ pop from queue
15:   else
16:      $e = (s, D)$  first in  $E_S \setminus seen$  ▷ pop from sorted  $E_S$ 
17:    $seen = seen \cup \{e\}$ 
18:    $nodes = \{d \in D \mid \rho(d) = p_{null}\}$  ▷ unassigned destinations only
19:   if  $inbound(s) = \emptyset$  then  $nodes = nodes \cup \{s\}$  ▷ include input nodes
20:   while  $|nodes| > 0$  do
21:      $n = argmin_{m \in nodes} (|inbound(m) \setminus apc|, -|inbound(m)|)$ 
22:     if  $npc = C_{npc}$  or  $spc > C_{spc}$  or  $|apc \cup inbound(n)| > C_{apc}$  then
23:       assert  $npc > 0$  ▷ else neuron  $n$  can't fit core constraints
24:        $pq(a) = 0 \forall a \in E_S$  ▷ clear the queue
25:        $p_{curr} = next-partition(P)$ 
26:        $npc = 0, spc = 0, apc = \{\}$ 
27:       continue
28:      $npc += 1, spc += 1, apc = apc \cup inbound(n)$ 
29:      $\rho(n) = p_{curr}$ 
30:      $nodes = nodes \setminus \{n\}$ 
31:     for  $c$  in  $(inbound(n) \cup outbound(n)) \setminus seen$  do ▷ visit h-edges
32:        $pq(c) = (pq(c) \cdot size(c) + 1) / (size(c) - 1)$  ▷ occurrences / size
33:        $size(c) -= 1$ 
34: assert  $|P| < |H|$  ▷ else exceeded the partition count limit
35: return  $\rho$ 

```

used to track the number of times each yet-unseen h-edge appears among those connected to the assigned nodes. In particular, the queue tracks, for each still unvisited h-edge, the ratio between the number of nodes assigned to the current partition that it touches and the h-edge's count of connected, but still unassigned, nodes (line 32). Such ratio is then scaled by the h-edge's spike frequency to become the priority (line 14). When the priority queue is not empty, it diverts the outer loop's behavior, making the next h-edge to be handled the one with the highest priority (line 13). Upon instantiation of a new partition, the queue is flushed (line 24). Execution terminates after having visited all h-edges.

Here, the queue's priority is an incremental proxy measure of second-order affinity; as such, it focuses on the spike-frequency-weighted fraction of co-membership exhibited in each remaining h-edge by the nodes in the current partition. Crucially, by this definition, the priority can be computed with only the limited extra burden of iterating once over each node's connections. The fallback on h-edges ordered by size, instead, exists to rapidly fill the queue while assigning as many nodes as possible early on, speeding up subsequent iterations. Then, our node selection policy within each h-edge strictly ensures maximum synaptic reuse while prioritizing a snug fit inside constraints; for this reason, it overlooks the spike frequency. This results in overall fuller partitions with high spike replication, while spike frequency returns

Algorithm 2 Greedy Nodes Ordering

Input: $G = (N, E, w)$ ▷ here G can be either G_S or G_P
Output: $ordered(N)$

```

1:  $L = ()$  ▷ empty ordered nodes list
2: for  $n$  in  $N$  do
3:    $pq(n) = 0$  ▷ addressable priority queue
4:    $inbound(n) = \{(s, D) \in E \mid n \in D\}$  ▷ precompute inbound sets
5:    $outbound(n) = \{(s, D) \in E \mid s = n\}$  ▷ outbound singletons
6: for  $n$  in  $argmin_{m \in N} (|inbound(m)|)$  do
7:    $pq(n) = +\infty$  ▷ start from min-inbound-set nodes
8: while  $|L| < |N|$  do
9:   if  $\exists m \in N \setminus L$  s.t.  $pq(m) > 0$  then
10:      $n = argmax_{m \in N \setminus L} pq(m)$  ▷ pop from queue
11:   else
12:      $n = argmin_{m \in N} (|inbound(m)|)$  ▷ default: min-inbound-set node
13:    $L = L \cup n$ 
14:   for  $e = (s, D)$  in  $outbound(n)$  for  $m$  in  $D$  do
15:      $pq(m) += w(e)$ 
16: return  $L$ 

```

to prominence when the next h-edge is extracted from the priority queue. In general, visiting an h-edge assigns only its destination nodes, since doing so for the source would require all its inbound h-edges to follow it in the current partition. But depending on the topology, a source might have a very different inbound set, easily saturating the partition, in turn winding up apart from its affine nodes. However, nodes that receive outside inputs don't present any inbound h-edge; as such, we are free to place them close to their destinations.

Even if this approach mainly iterates over h-edges, that is but a means to visit and assign nodes in an order that favors affinity. In fact, each node is processed, and its connections visited, only once (line 31), giving a complexity of $O(n \cdot h \cdot \log(C_{npc} \cdot h))$. Where the logarithm is the cost of managing the heap implementing the priority queue, that, being reset with each partition, can contain at most h h-edges per neuron in the partition. Thus, by hiding the relatively small constant C_{npc} , expanding the definition of h , and using $n = e$, we can rewrite the complexity as $O(e \cdot d \cdot \log d)$.

IV-A3. Sequential Partitioning: A fast strategy from [7] that requires nodes to be ordered, it then puts successive nodes in the same partition as long as NMH constraints allow for it, and only upon violating them does it initiate a new partition. Clearly, this becomes effective only if nodes can be ordered such that successive ones have almost identical connection patterns with the rest of the h-graph. The nodes' overlapped connections, in turn, would lead to synaptic reuse and hardware resource sharing, requiring fewer partitions and lowering the partitioning's connectivity cost.

Obtaining such a convenient ordering is, however, just as hard as solving the partitioning problem. Therefore, we adopt two strategies. For ANN-derived SNNs, nodes are constructively ordered by iterating through the ANN's layers and sequencing the neurons inside each layer. Hence, the resulting order will naturally present similar inbound and outbound h-edges among neighboring nodes [7]. For example, consider the overlap between the receptive fields of two neighboring output neurons in a convolution. While, for arbitrary SNNs, no such order is intrinsically available. Ergo, it needs to be constructed, but finding an optimal one that clusters nodes with highly overlapping inbound

connectivity – maximizing synaptic reuse between nearby nodes in the ordering – reduces to a variant of the minimum linear arrangement problem, which is itself NP-hard [28]. Consequently, we settle for a greedy approximation: we keep an addressable priority queue of nodes, initially filled with those having the least inbound h-edges (usually those with none), all with the highest priority. We then iteratively build the order by removing the queue’s top element and adding all the nodes it is connected to to the queue, each with priority equal to the spike frequency of its connection. If a node is already present in the queue, its priority is increased by such spike frequency. Refer to Alg. 2 for details¹. This approach yields an ordering with high local synaptic reuse by repeatedly grabbing the next most connected node to those already seen, exactly what sequential partitioning expects to exploit. Its complexity is $O(n \log n \cdot h) = O(e \cdot d \cdot \log n)$; the logarithm emerges from implementing the queue as a heap.

In the end, sequential partitioning is most effective when the input SNN has a layered-like structure, but it can be efficiently generalized to any SNN. In particular, once the ordering is available, this is the fastest partitioning technique we hereby consider, with complexity $O(n)$.

B. Initial Placement

The placement determines which SNN partition is assigned to which NMH core. Since the interconnect of NMH propagates spikes along rows and columns, the objective is to minimize the Manhattan distance between heavily connected partitions. Heavily connected partitions striving to get closer means that their common h-edges will tend to remain confined within a small neighborhood of cores, which is the aforementioned concept of connections locality.

The purpose of an initial placement is to provide a good starting point for placement heuristics by attempting to maximize the locality of cores that share many h-edges. Not all placement routines can leverage an initial placement, but when that is possible, providing a good one is crucial. To this end, we consider two algorithms that preserve locality while laying h-graph nodes on a lattice. First, the discrete Hilbert Space-filling Curve [7]. Second, the use of spectral embeddings to project the h-graph structure into 2D Euclidean space. An example of both is given in Fig. 5.

IV-B1. Hilbert Space-filling Curve: The discrete Hilbert space-filling curve provides a mapping from a 1D sequence to 2D integer coordinates. Moreover, it does so while preserving the sequence’s locality, as in neighboring elements in the sequence are mapped to points in close spatial proximity [7]. Assuming to have a sequence of nodes with high connections locality, this gives us an easy transition from 1D to 2D, resulting in a good initial definition for γ . But of course, to preserve locality, we need to have locality in the first place.

Unlike the original SNN, its partitioned h-graph is likely to present cycles, and we can’t know a priori if its nodes will present some natural order with strong locality. Therefore, we employ the same ordering techniques discussed in Sec. IV-A3 for arbitrary SNNs. When the partitioned h-graph is acyclic, we construct its topological ordering using a

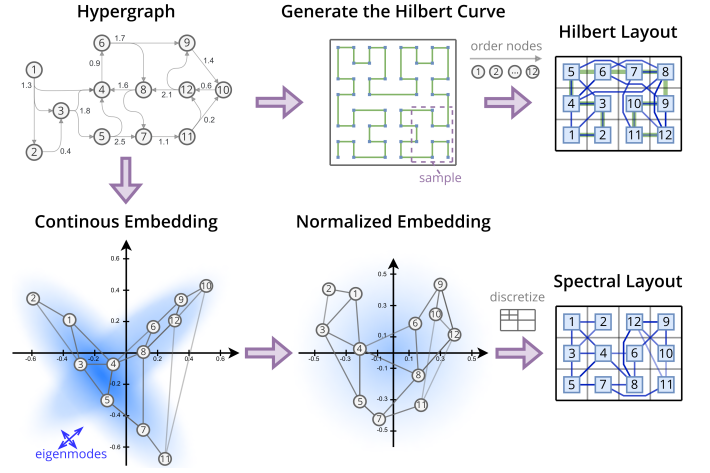


Fig. 5: Example constructions of Hilbert and spectral placements.

queue-based variant of Kahn’s algorithm [31], where roots are enqueued first and, at each step, outgoing h-edges are processed in decreasing weight order before newly freed nodes are added to the queue. This is typically the case of layered SNNs that underwent sequential partitioning, as in [7]. Otherwise, our greedy order from Alg. 2 is used. Since constructing the Hilbert curve has linear cost in the number of coordinates spanned, this method’s final complexity is $O(e \cdot d)$ for acyclic graphs, else $O(e \cdot d \cdot \log n)$, as in Sec. IV-A3.

IV-B2. Spectral Placement: To compute an initial compact and structure-aware placement of an h-graph onto a discrete 2D lattice, we can rely on the h-graph’s spectrum, in the form of the eigenvalues and eigenvectors of its Laplacian matrix. The idea is to first project the h-graph’s connectivity geometry into a two-dimensional coordinate system, using the lowest nontrivial eigenmodes of the Laplacian as a guide. Then, we just need to scale and discretize such an embedding over our lattice. The so-obtained layout captures both first- and second-order affinity: the Laplacian-induced quadratic smoothness objective simultaneously causes strongly connected nodes to collapse together and penalizes the variance within each h-edge [21], [32].

Concretely, we construct the normalized Laplacian of the hypergraph $\mathcal{L} \in \mathbb{R}^{|P| \times |P|}$ [32] by exploding each h-edge in individual connections:

$$\mathcal{L}_{i,j} := \begin{cases} 1 & \text{if } i = j \\ -\frac{1}{\sqrt{wdeg(i) wdeg(j)}} \sum_{\substack{e=(s,D) \in E_P \\ \{i,j\} \subseteq \{s\} \cup D}} w_P(e) & \text{if } i \neq j \end{cases} \quad (8)$$

$$\text{where } wdeg(p) = \sum_{\substack{e=(s,D) \in E_P \\ p \in \{s\} \cup D}} w_P(e).$$

From it, we compute the two eigenvectors corresponding to the smallest non-zero eigenvalues [33]:

$$(\mathbf{u}_1, \mathbf{u}_2) \text{ s.t. } \mathcal{L} \mathbf{u}_k = \lambda_k \mathbf{u}_k, \quad 0 < \lambda_1 \leq \lambda_2 \leq \dots, \quad (9)$$

The resulting eigenmodes provide the smoothest nontrivial orthogonal directions minimizing the quadratic objective:

$$Q(\Gamma) := \text{tr}(\Gamma^\top \mathcal{L} \Gamma) \text{ where } \Gamma = [\gamma_c(p)]_{p \in P}, \quad (10)$$

which encodes a penalty proportional to the dispersion

within each h-edge – not just pairwise links, but whole h-edge spread – and, as such, it precisely boosts connections locality. These two eigenvectors (Γ columns) each have one entry per node and together define a 2D coordinate for every node, one component from each eigenvector (Γ rows). So:

$$\Gamma = [\mathbf{u}_1, \mathbf{u}_2], \text{ and thus } \gamma_c(p) = (\mathbf{u}_{1,p}, \mathbf{u}_{2,p}) \quad (11)$$

is our initial continuous embedding, with $\gamma_c : P \rightarrow \mathbb{R}^2$ being the continuous dual of γ . This γ_c is first normalized to fit within the unit square, then scaled to occupy a compact, nearly-square, rectangular region encompassing enough points of H to fit all partitions. The compact layout is centered within the grid, and each node's continuous position is then discretized to the nearest unoccupied integer lattice point, at last yielding γ . To ensure that each node is assigned a unique coordinate without collisions, a KD-tree is used to efficiently search for the nearest available grid point, and assigned points are removed from the candidate set as placement proceeds. Nodes are visited in descending order of total spike frequency for their bound h-edges.

The final result is a γ to grid coordinates that preserves the structural affinities of the partitioned h-graph while minimizing the total edge length under Manhattan distance. If the Laplacian is represented in sparse form, computing the k smallest nonzero eigenpairs with an iterative method requires $O((n + e \cdot d) \cdot k)$ operations over a small number of iterations [34]. Here, $k = 2$, and $e \cdot d$ is the number of non-zeros in the Laplacian. Then, normalization and scaling are $O(n)$, and the discretization step is $O(n \log n)$ due to nearest-neighbor search with a KD-tree. Overall: $O(e \cdot d + n \log n)$.

C. Placement Refinement

With a good initial placement, costly refining algorithms can be efficiently applied on a local basis to improve it. We take one such algorithm from literature [7]. Then, we also include an algorithm from [11] that directly goes from h-graph to placement, with no need for an initial solution. Fig. 6 depicts the basic ideas of both algorithms.

IV-C1. Force-Directed: Given an initial placement, the idea is to efficiently identify pairs of neighboring partitions such that if their placements were swapped, connections locality would improve. In [7] it is proven that minimizing, across all partitions, a virtual potential of G_P and γ :

$$Pot_{\gamma}(p) := \sum_{e=(s,D) \in E_P \mid p \in D} \|\gamma(p) - \gamma(s)\| \cdot w_P(e), \quad (12)$$

is equivalent to minimizing the total system energy and latency as defined in Table I. This leads to defining a force as the delta in the potential when a partition is moved (not yet a swap, temporary overlaps are accepted at this point) by one core along one of the four cardinal directions:

$$\begin{aligned} Force_{\gamma}(p, v) &:= Pot_{\gamma}(p) - Pot_{\gamma'}(p), \\ \text{where } \gamma'(q) &= \begin{cases} \gamma(q) & \text{if } q \neq p \\ \gamma(p) + v & \text{if } q = p \end{cases} \\ \text{and } v &\in \{(1, 0), (-1, 0), (0, 1), (0, -1)\}. \end{aligned} \quad (13)$$

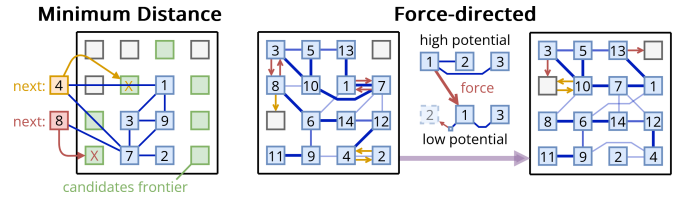


Fig. 6: Overview of the considered placement refinement algorithms.

As a result, a placement can be optimized by swapping partitions between neighboring cores so long as the sum of opposing forces pulling on the two is positive.

The refinement routine, adapted from [7], simply iterates on all pairs of neighboring cores whose partitions give a positive sum of forces, swaps them, and repeats, until no such pairs remain. Candidate pairs are visited by higher-force first. Recomputing all forces after a change is costly; thus, they are lazily updated only on candidate partitions before swapping their placements and on all partitions connected to swapped ones. We improve this technique by not only attempting swaps between placed partitions, but between any unused core and surrounding ones with a placed partition as well. In Eq. 13, this means that $\gamma(p) + v$ only needs to be valid – inside H – and not necessarily in γ 's codomain already. In practice, this change allows the placement's active cores to change when not all cores are utilized. Then, we modify the evaluation of Eq. 12 to use $\max(\|\cdot\|, 1)$ instead of $\|\cdot\|$, to ensure that when calculating deltas in the potential, any co-located partitions still present a unit Manhattan distance. Otherwise, their force's effect would be neglected, potentially causing endless loops of positive-force swaps.

One iteration of the above routine has costs $O(n \cdot d \cdot h^2)$, since each force calculation is linear in h and a swap can cause the update of up to $2 \cdot d \cdot h$ forces among other partitions. Assuming t iterations, the final complexity is $O(t \cdot n \cdot d \cdot h^2)$. That said, t is hard to estimate a priori, with our experiments seeing it vary between 50 and 1.5k. Still, refinement can be terminated early if the achieved performance is deemed sufficient, exposing t as a parameter.

IV-C2. Minimum Distance Placement: This is the placement algorithm used in TrueNorth [11]. Input partitions, those receiving spikes from outside the system, are spread out as much as possible while remaining centered and evenly spaced between themselves and the cores' lattice's borders. Then, each subsequent partition is placed on the core that minimizes its total Manhattan distance from all already-placed partitions it is connected to. This is performed while going through partitions in topological order, ensuring that all sources of h-edges inbound to the current node have already been placed, thus making the raw Manhattan distance a viable proxy for the final mapping performance. Once again, we use a queue-based variant of Kahn's algorithm [31] to obtain the topological order for acyclic h-graphs and our greedy techniques presented in Alg. 2 when the partitioned graph is cyclic. Such a visit of nodes works since, for the Manhattan distance to approximate mapping performance, it is enough to see the highest spike frequency h-edges.

The way it was presented in [11], this method required that the SNN be trained, from scratch, as if already partitioned;

	Network	Node count	Connections count	Mean h-edge cardinality	Target constraints
layered / feedforward	16k_model	20k	766k	37.3	small
	64k_model	110k	23M	210.3	small
	256k_model	216k	90M	417.2	large
	1M_model	302k	256M	848.1	large
layered / feedforward	LeNet	14k	875k	63.2	small
	AlexNet	208k	145M	696.2	large
	VGG11	194k	133M	688.3	large
	MobileNet V1	6.9M	577M	83.5	large
cyclic	Allen V1	231k	70M	304.7	large
	16k_rand	2^{14}	2.1M	128	small
	64k_rand	2^{16}	12.6M	192	small
	256k_rand	2^{18}	67.4M	256	small

TABLE III: Spiking neural networks used in the experiments.

Algorithm	Partitioning	Initial Placement	Placement Refinement
	hierarchical (IV-A1)	Hilbert curve (IV-B1)	force-directed (IV-C1)
	hyperedge overlap (IV-A2)	spectral (IV-B2)	
	sequential (IV-A3)	minimum distance (IV-C2)	

TABLE IV: Algorithms forming the compared mapping techniques.

we took the liberty to generalize it to arbitrary SNNs by pairing it with other partitioning algorithms. Moreover, we found two ways to improve this algorithm over its original version. First, we can weight each Manhattan distance by the total spike frequency of the h-edges connecting the two partitions. Second, rather than searching all cores for the one of lowest total Manhattan distance, we look exclusively at those immediately adjacent to cores that are already in use. This means that to place a partition, we just need to visit the frontier around γ 's growing codomain.

This is a very straightforward technique that has a compelling complexity of $O(n \cdot h \cdot |H| + q)$. Here, $n \cdot h$ is the cost of computing the distance from the present partition to all those it is connected to, that could all already be placed, and q is either $O(e \cdot d)$ for acyclic graphs, else $O(e \cdot d \cdot \log n)$, for the topological ordering. Although restricting the next partition's candidate placements to the active core's frontier does not reduce the asymptotic complexity, it makes the contribution of $|H|$ negligible in practice. Combined with its linear dependence on n and h , this makes the minimal distance placement the most scalable across our experiments.

V. EXPERIMENTAL EVALUATION

A. Experimental Setup

The algorithms we considered are summarized in Table IV. Our comparison spans all possible combinations of such partitioning, initial placement, and placement refinement techniques. Our main baseline is formed by sequential partitioning and Hilbert curve placement with force-directed refinement. For partitioning, however, we also compare against EdgeMap [15] and include the original unordered sequential partitioning from [7], which solely relies on the intrinsic order of nodes in the network, if any. Target NMH parameters have been introduced in Tab. II, we will rely on the "small" configuration up to 2^{26} connections, then switch to the "large" one. This was required as bigger models readily exceed 4096 inbound connections per neuron when h-edge cardinality grows in the hundreds. Relevant quality metrics have been defined in Tab. I; additionally, we will use the Energy-

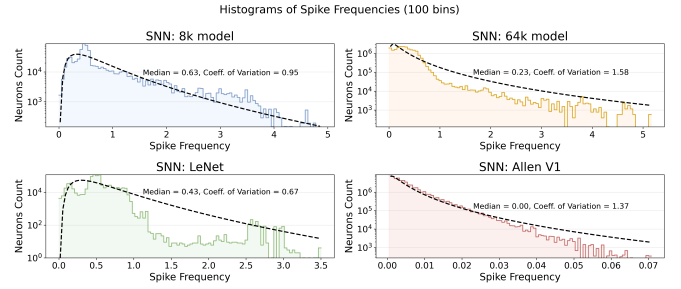


Fig. 7: Spike frequencies distributions for four selected SNNs, fitted by a log-normal probability density function.

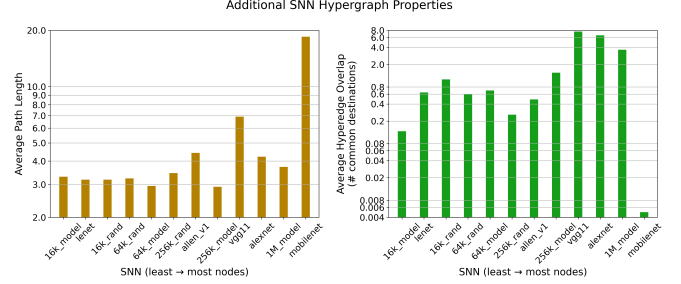


Fig. 8: Average path length and h-edge overlap of considered SNNs.

Latency Product (ELP) as a compound indicator of mapping performance. While these metrics are not representative of any specific hardware platform, they have already been effectively used to rank mappings [7], [15], [16].

As anticipated, existing tools that implement heuristics from Sec. IV-A1 lack support for the constraints imposed by NMH. At the same time, recent SNN mapping tools [7], [15] don't have available artifacts. Therefore, we re-implemented all presented heuristics, exploiting our h-graph-based model's advantages whenever possible and ensuring compliance with NMH constraints. This was done in Python 3.13 and with no explicit use of parallelism. Experiments were carried out on an AMD EPYC 7453 @ 2.75GHz with 256GB of RAM.

The SNNs involved in our experiments are reported in Table III. For layered SNNs, we considered eight convolutional neural networks. They include four custom-built ones labeled "x_model", with x being the number of parameters. Their architecture stacks VGG-like [35] blocks until the desired number of parameters is reached, followed by global average pooling and a dense layer. Then, we have four networks from literature: LeNet, AlexNet, and VGG11 trained on the Cifar10 dataset, and MobileNet for ImageNet 2012, all as implemented in [35], [36], [37]. We used SNNToolBox [22] to generate SNN versions of all networks and measure their spike frequencies while running inferences on 10% of their respective datasets.

As representatives of SNNs with arbitrary connectivity, we instead considered four biologically-inspired and cyclic networks. First, we have the Allen V1 [38], a SNN modeled after a mouse's primary visual cortex. Continuing, we took inspiration from liquid state machines [18] and feedback SNNs [25] to randomly generate three h-graphs based on their topology. These are named "x_rand", x being the number of nodes. To construct them, we instantiate the required amount of nodes and randomly assign every one to a pair of coordinates inside the unit square. Then, for each node, we sample its prospective connections count from a Poisson distribution

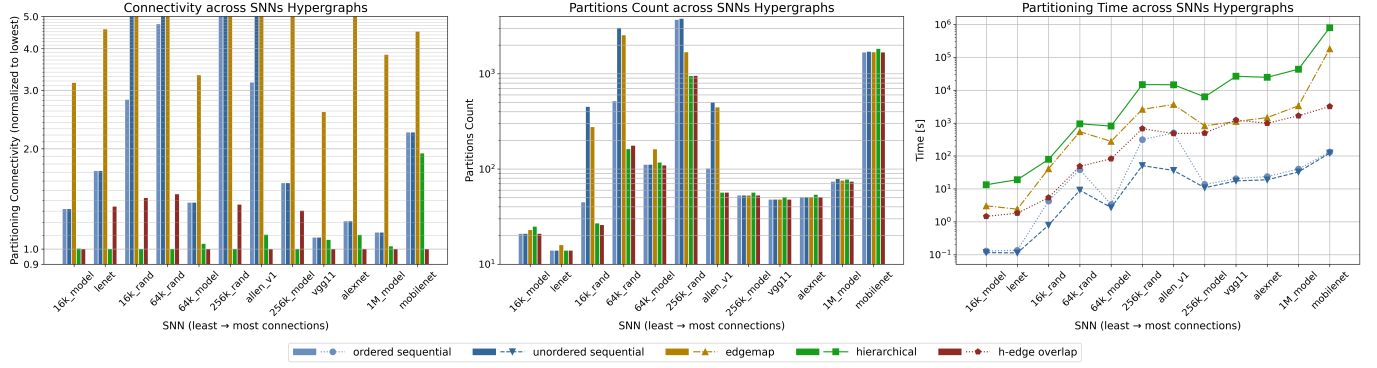


Fig. 9: Connectivity and execution time comparison between the considered partitioning heuristics.

of expected value equal to a fixed mean h-edge cardinality. The target neurons of each h-edge follow a distance-dependent probability, where the likelihood of connection decays exponentially with the Euclidean distance over the previously assigned coordinates. Finally, spike frequencies are sampled from a log-normal distribution with median 0.23 and coefficient of variation 1.58. This choice is supported by biological evidence [39], but also matches the spike frequency distribution of our other models, as shown in Fig. 7. The resulting dense topology of strong connections is meant as a spike in difficulty – pun intended – for mapping.

Fig. 8 shows the average path length between any two nodes on the considered SNNs. Such slow-growing values highlight the fact that both layered and cyclic SNNs are small-world networks. It follows that, as the figure also shows, any pair of h-edges tends to overlap quite often, providing the basis for the exploitation of synaptic reuse. As a control, our randomly-generated h-graphs naturally fit among others for both measures. MobileNet, instead, is a slight outlier, mainly due to its original ANN’s high layer count, and shall serve as a test to see how well algorithms behave when reuse opportunities are scarce.

B. Results Discussion

In Fig. 9, we report results for partitioning algorithms, compared by the achieved connectivity (Eq. 7) and number of partitions. Then, with Fig. 10, we show our results for complete mappings, after performing both partitioning and placement.

V-B1. Partitioning Results: Looking at execution time first, we can clearly see three trends matching the presented algorithmic complexities. All algorithms grow linearly with the count of nodes or h-edges. At the bottom, unordered sequential partitioning is the fastest, having no further dependency. Forming the middle line, with a linear dependency on h-edge cardinality, we have partitioning by h-edge overlap, EdgeMap, and ordered sequential (on cyclic h-graphs). All those algorithms iterate over each node’s connections, or each h-edge’s connectees, to rely upon guidance from either first- or second-order affinity. The upper trend is constituted by hierarchical partitioning, with a quadratic dependency on h-edge cardinality. In contrast to the growing number of neurons dictating the trend in execution time, the h-edge cardinality is relatively small and poses little limits to scalability, averaging at 316.4 on our SNNs. Yet, being a multiplicative

factor, it can be decisive in slowing down execution from hours to days. We shall now evaluate whether each additional computational resources investment pays off in connectivity.

Hierarchical partitioning dominates for small h-graphs, but the load of edge-coarsening soon hinders its execution time. On average, its connectivity cost is $0.47\times$ that of sequential partitioning, and $0.95\times$ that of our overlap method.

Sequential partitioning, while simple, performs well on layered networks. In spite of its lack of any active form of guidance, at times it reaches within just $1.02\times$ higher connectivity compared to the hierarchical method. Even then, it is exceedingly fast and scalable, albeit so long as nodes are already ordered. When arbitrarily connected networks are considered, however, it has to rely on our greedy ordering scheme to retain its quality of results. This increases its execution time proportionally to h-edge cardinality, making it run in the same time as our overlap-based method, which nevertheless finds partitionings with at least $0.51\times$ the cost. Foregoing such additional ordering step, the unordered sequential partitioning variant remains fast, but inevitably incurs up to $11.4\times$ its counterpart’s connectivity.

Our new h-edge overlap-driven approach fits precisely between the slow hierarchical and fast sequential techniques under all metrics. Its time spent on evaluating second-order affinity pays off, as it always achieves a connectivity within $0.52\times$ and $1.46\times$ that of hierarchical partitioning and $0.32\times$ to $0.91\times$ that of sequential. On occasions, it even manages to produce up to 5% fewer partitions than other methods, thanks to better exploitation of synaptic reuse. Its execution time scales well on large networks, being within the expected complexity factor of h-edges cardinality – at most $28.2\times$ – slower than our baseline. Moreover, on cyclic networks, our algorithm outspeeds all others but unordered partitioning; its results always reaching within $1.28\times$ of the best.

With both hierarchical partitioning and our overlap method leaning heavily on second-order affinity, we can trace back their small gap in results to a major difference. The first repeatedly evaluates the affinity between each pair of nodes, as it builds all partitions simultaneously, thus managing to hide connections more uniformly. The second measures affinity only locally to the present partition, as it fills them sequentially. Even so, it still benefits from second-order affinity in ordering visits to nodes and h-edges, in turn only needing to scan each connection once.

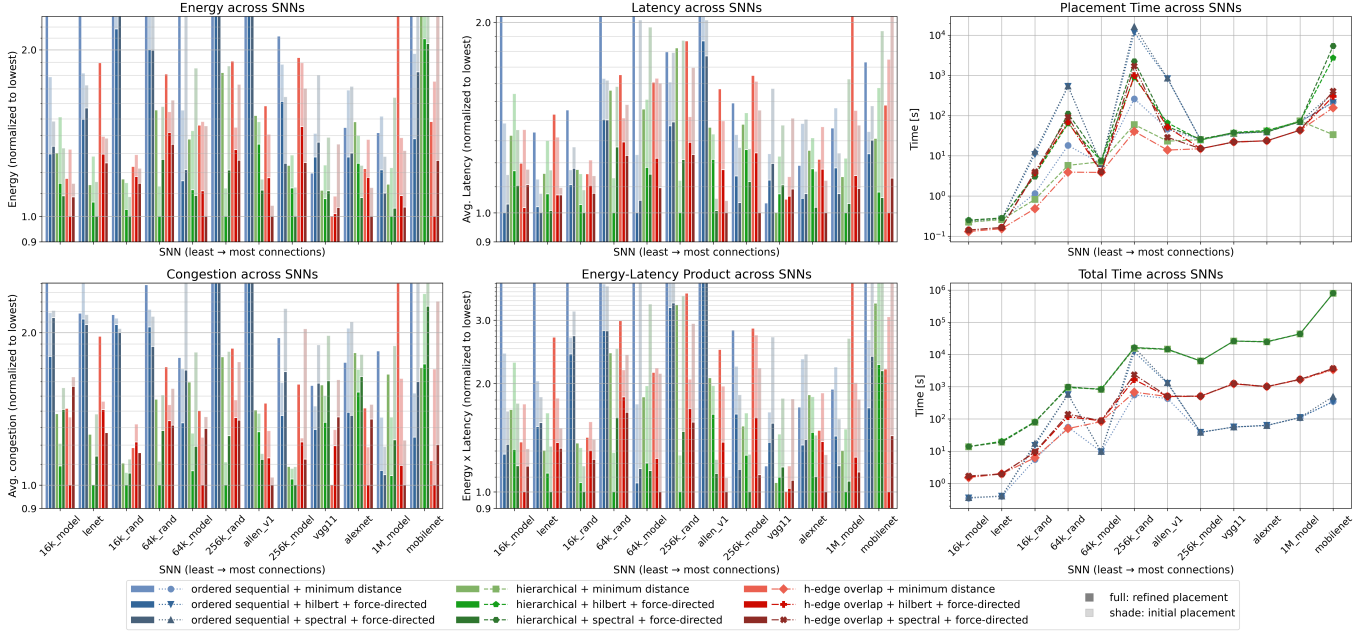


Fig. 10: Mapping performance and construction time comparison between all placement techniques and partitioning algorithm pairs.

The node-centric, graph-based scheme of EdgeMap, which relies foremost on source-destination connection strength, serves as our control experiment. Iterating over each connection of each node, its execution time is comparable to our approach, however, its achieved connectivity is $8.5\times$ worse than ours, on average. This highlights how second-order affinity results in significantly better guidance compared to individual connections, for a similar computational cost.

V-B2. Placement Results: The differences in connectivity achieved by partitioning algorithms are largely reflected in their final mapping metrics, regardless of the placement algorithm used. The choice of partitioning algorithm also doesn't seem to slow down placement, aside from minor variations attributable to slight gaps in partitions count. So, let us consider for now the best mapping obtained for each partitioning scheme. On average, hierarchical partitioning produces mappings with ELP $0.98\times$ relative to our method. In turn, our overlap partitioning bests sequential with a mean $0.63\times$ ELP. Across individual metrics, our method is always within $1.35\times$ of the best while consistently reaching values between $0.09\times$ and $1.01\times$ of any faster method. In particular, for the Allen V1, our overlap partitioning plus refined spectral placement mapping technique unilaterally finds the best mappings in the least time compared to all other solutions. This result underlines the key role that our identified affinities plays in handling the challenge of realistic SNNs. Then again, on MobileNet, the largest network we considered, overlap-based partitioning also leads in mapping quality, solidifying its consistency at scale.

For initial placements, the Hilbert space-filling curve performs well, especially with few partitions or high path length networks (e.g. VGG11, MobileNet), where even far-apart cores are not excessively penalized. However, its full reliance on nodes ordering starts to show limits on densely connected and cyclic SNNs. Our proposed spectral layout is instead agnostic w.r.t. the number of cores, as it is driven

directly by the affinities between nodes. Consequently, after refinement, the spectral method gives, on average, $0.96\times$ the ELP, improving to $0.91\times$ across our four shortest path length networks. On the 256k_model, where the path length is at its shortest, spectral further dominates with a mean $0.73\times$ energy and latency relative to Hilbert across all partitionings. Nonetheless, looking at the interconnect's congestion shows that the Hilbert curve's mean resulting traffic is $0.92\times$ the spectral method's. We ascribe this to communication being more evenly distributed over core-to-core links as a side-effect of the curve's locality-preserving properties. That is in contrast to the spectral layout's many connections passing through the lattice's center following the spectrum's normalization during construction. Both algorithms require negligible execution time, and neither has a significant impact on the execution time of the subsequent refinement.

Comparing the refined layouts against their initial versions from Hilbert and spectral placement, we see that force-directed refinement consistently lowers every metric to $0.51\text{--}0.87\times$ its initial value. Execution time for refinement, in general, grows linearly with the number of partitions that are being placed. For most of our experiments, it usually is less than two minutes, as we tend to have <100 partitions, thus making partitioning time dominate. Yet, when networks are dense enough for partitions to reach in the hundreds, mainly on our small NMH configuration, refinement starts occupying a significant part of the mapping process. Surely, there is always the option to interrupt refinement early if, by then, results are deemed satisfactory, but we leave that up to the final user. On the other hand, minimum distance placement starts to emerge as a faster alternative. Its performance metrics are always within $2.18\times$ of the best achieved by force-directed refinement and $0.73\text{--}1.46\times$ of both initial placements. However, it consistently runs in less than $2k$ seconds. Ultimately, force-directed refinement is the most flexible option, especially if a very good initial placement is

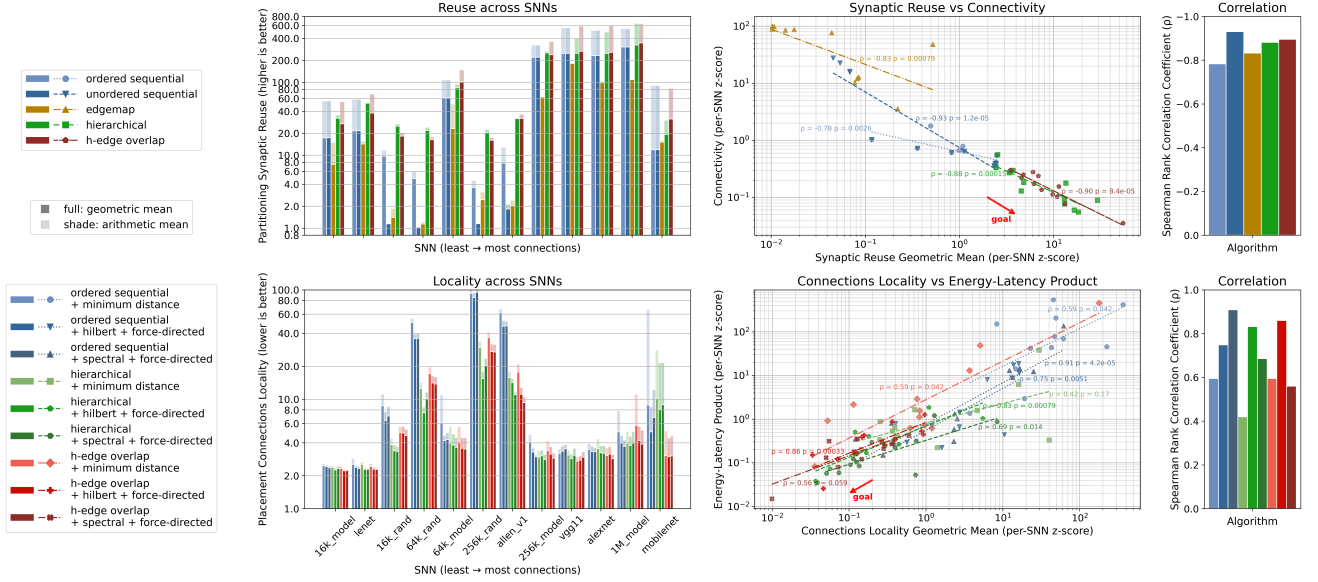


Fig. 11: Synaptic reuse and connections locality measures and their correlation with mapping connectivity and energy-latency product.

available, but greedier alternatives will be faster at scale.

We thus conclude that relying on hypergraphs, their properties, and node affinities, be it with hierarchical partitioning, h-edge overlap, or spectral placement, yields very promising results, especially as SNN sizes grow and topologies vary. We find that the regime of linear complexity in the SNN’s connections, where affinities remain observable and actionable, holds particular potential for high-quality partitioning at scale. Accordingly, the hierarchical method dominates for smaller SNNs, while overlap-based partitioning scales more gracefully; however, specific choices depend on available time and desired mapping quality. Placement has more margin due to its smaller problem size, and running an ensemble of different techniques on a time limit – then selecting the best final mapping – is practicable.

C. Verifying Mapping Properties

To empirically verify our intuitions from Sec. III-A, we hereby define and measure two quantities representing synaptic reuse and connections locality. We quantify synaptic reuse as the mean total number of individual inbound connections (synapses) per partition over the number of distinct inbound h-edges (axons) (partly based on [16]):

$$SR(G_S, G_P, \rho) := \text{mean}_{p \in P} \left(\frac{\sum_{(s,D) \in E_S} |\{d \in D \mid \rho(d) = p\}|}{|\{(s,D) \in E_S \mid \exists d \in D, \rho(d) = p\}|} \right). \quad (14)$$

Continuing, we quantify connections locality as the average number of lattice points enclosed by the convex hull (denoted by *conv*) defined around the cores connected by each h-edge:

$$CL(G_P, H, \gamma) := \text{mean}_{e=(s,D) \in E_P} |conv(\{\gamma(p) \mid p \in \{s\} \cup D\}) \cap H|. \quad (15)$$

Fig. 11 reports their measures across our experiments. Data are presented using both geometric and arithmetic means as *mean*. For synaptic reuse, the arithmetic mean reflects aggregate overlap, while the geometric mean emphasizes consistency across partitions and heavily penalizes low-

overlap partitions. Similarly, for connections locality, the arithmetic mean captures the overall average spatial footprint of h-edges, while the geometric mean emphasizes the typical footprint and is less influenced by a few very large h-edges.

Qualitatively, the trend for synaptic reuse tracks the SNNs’ average hyperedge overlap, while connections locality is inversely related to the average path length (refer to Fig. 8). Then, all per-SNN performance differences between techniques are closely reflected by the hereby measures. More in detail, we find that synaptic reuse’s geometric mean perfectly follows connectivity, while the arithmetic mean tends to diverge. Looking at our overlap-based algorithm, it often exhibits a noticeably higher arithmetic mean, implying that while it can reuse more synapses, it doesn’t do so evenly across partitions, leading to more costly cuts. Instead, the geometric mean reflects hierarchical partitioning’s dominant results, suggesting that a uniform exploitation of synaptic reuse is preferable, albeit tougher to achieve. For connections locality, there is little deviation between arithmetic and geometric mean, indicating that the spread of h-edges is mostly homogeneous. Moreover, both means clearly reflect final mapping energy, latency, and congestion.

To assess the relationship between reuse/locality and mapping connectivity/ELP, Fig. 11 also plots them against each other. We then analyze the monotonic association between them using the Spearman’s rank correlation, with the resulting coefficients shown in the figure. Because different SNNs exhibit widely different quality and property values, we here standardized both metrics per h-graph (z-score).

Our conjectures, forming the basis for the present study, were that exploiting synaptic reuse leads to good partitionings and that improving connections locality provides better placements. Those are indeed confirmed by the evident correlation between synaptic reuse and connectivity, and between connections locality and mapping performance. Across all partitioning techniques, Spearman’s rank correlation between reuse and connectivity is strongly negative ($\rho \approx -0.86$ on average) with small deviation, proving higher synaptic

reuse values consistently correspond to better partitions. For placement, all correlations are significantly positive ($\rho \approx 0.69$ on average), confirming that mappings achieving lower ELP also exhibit better connections locality. These consistent monotonic relationships demonstrate that our measures in Eqs. 14 and 15 are reliable predictors of solution quality.

We can consequently reaffirm that uniformly pursuing synaptic reuse and connections locality is a sound approach to raise mapping quality. Additionally, these concepts hold firmly as the network is scaled or its topology is diversified.

VI. CONCLUSION

Bringing together nodes that partake in the same hyperedges emerges as the key policy identified in this work, one that we have shown to correlate strongly with high-quality mappings of spiking neural networks on neuromorphic hardware. This principle benefits both partitioning – by harnessing synaptic reuse – and placement – by increasing connections locality. Taken together, these findings firmly cement hypergraphs as a sound and relevant abstraction for SNNs throughout their mapping, capable of naturally and precisely capturing the replication of spikes and their spread across cores.

Thereby, we opened a new avenue for high-quality SNN mapping based on hypergraphs. Our preliminary selection of algorithms already demonstrates the practical value of this direction, delivering mappings up to twice as efficient as state-of-the-art graph-driven heuristics and remaining effective across layered, recurrent, and biologically plausible SNNs. Notable gains come from hyperedge overlap-based partitioning, which scales linearly with problem size, and spectral initial placement, prized for its versatility. These results indicate that hypergraph information can be exploited further while keeping computation scalable – an essential requirement for approaching networks with billions of neurons.

To foster the development and comparison of further hypergraph algorithms applied to SNNs mapping, we plan to release our benchmark hypergraphs and algorithm implementations as open source. At last, our future research efforts will focus on refining these hypergraph-based mapping heuristics and on the multi-chip generalization of the mapping problem. We trust that this line of work lays the foundation for the continued improvement of mapping tools, paving the way for brain-scale SNNs on neuromorphic hardware.

REFERENCES

- [1] F. Akopyan *et al.*, “Truenorth: Design and tool flow of a 65 mw 1 million neuron programmable neurosynaptic chip,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 34, no. 10, pp. 1537–1557, 2015.
- [2] M. Bouvier, A. Valentian, T. Mesquida, F. Rummens, M. Reyboz, E. Vianello, and E. Beigne, “Spiking neural networks hardware implementations and challenges: A survey,” *J. Emerg. Technol. Comput. Syst.*, vol. 15, no. 2, Apr. 2019. [Online]. Available: <https://doi.org/10.1145/3304103>
- [3] J. D. Nunes, M. Carvalho, D. Carneiro, and J. S. Cardoso, “Spiking neural networks: A survey,” *IEEE Access*, vol. 10, pp. 60 738–60 764, 2022.
- [4] M. Davies *et al.*, “Loihi: A neuromorphic manycore processor with on-chip learning,” *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.
- [5] S. B. Furber, F. Galluppi, S. Temple, and L. A. Plana, “The spinnaker project,” *Proceedings of the IEEE*, vol. 102, no. 5, pp. 652–665, 2014.
- [6] B. V. Benjamin, P. Gao, E. McQuinn, S. Choudhary, A. R. Chandrasekaran, J.-M. Bussat, R. Alvarez-Icaza, J. V. Arthur, P. A. Merolla, and K. Boahen, “Neurogrid: A mixed-analog-digital multiprocessor system for large-scale neural simulations,” *Proceedings of the IEEE*, vol. 102, no. 5, pp. 699–716, 2014.
- [7] O. Jin, Q. Xing, Y. Li, S. Deng, S. He, and G. Pan, “Mapping very large scale spiking neuron network to neuromorphic hardware,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 419–432. [Online]. Available: <https://doi.org/10.1145/3582016.3582038>
- [8] C. Fiduccia and R. Mattheyses, “A linear-time heuristic for improving network partitions,” in *19th Design Automation Conference*, 1982, pp. 175–181.
- [9] D. Papa and I. Markov, “Hypergraph partitioning and clustering,” *Handbook of Approximation Algorithms and Metaheuristics*, 05 2007.
- [10] G. Karypis, R. Aggarwal, V. Kumar, and S. Shekhar, “Multilevel hypergraph partitioning: Applications in vlsi domain,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 7, no. 1, pp. 69–79, 1999.
- [11] J. Sawada *et al.*, “Truenorth ecosystem for brain-inspired computing: scalable systems, software, and applications,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’16. IEEE Press, 2016.
- [12] C.-K. Lin, A. Wild, G. N. Chinya, T.-H. Lin, M. Davies, and H. Wang, “Mapping spiking neural networks onto a manycore neuromorphic architecture,” in *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2018. New York, NY, USA: Association for Computing Machinery, 2018, p. 78–89. [Online]. Available: <https://doi.org/10.1145/3192366.3192371>
- [13] A. Balaji, A. Das, Y. Wu, K. Huynh, F. G. Dell’Anna, G. Indiveri, J. L. Krichmar, N. D. Dutt, S. Schaafsma, and F. Catthoor, “Mapping spiking neural networks to neuromorphic hardware,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 1, pp. 76–86, 2020.
- [14] S. Song, H. Chong, A. Balaji, A. Das, J. Shackelford, and N. Kandasamy, “Dfsynthesizer: Dataflow-based synthesis of spiking neural networks to neuromorphic hardware,” *ACM Trans. Embed. Comput. Syst.*, vol. 21, no. 3, May 2022. [Online]. Available: <https://doi.org/10.1145/3479156>
- [15] J. Xue, L. Xie, F. Chen, L. Wu, Q. Tian, Y. Zhou, R. Ying, and P. Liu, “Edgemap: An optimized mapping toolchain for spiking neural network in edge computing,” *Sensors*, vol. 23, no. 14, 2023. [Online]. Available: <https://www.mdpi.com/1424-8220/23/14/6548>
- [16] C. Xiao, X. He, Z. Yang, X. Xiao, Y. Wang, R. Gong, J. Tie, L. Wang, and W. Xu, “Hierarchical mapping of large-scale spiking convolutional neural networks onto resource-constrained neuromorphic processor,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 5, pp. 1442–1455, 2024.
- [17] W. Severa, F. Wang, Y. Ho, F. Rothganger, A. Daram, and E. Gonzalez, “Benchmarking spiking network partitioning methods on loihi 2,” in *Proceedings of the Great Lakes Symposium on VLSI 2025*, ser. GLSVLSI ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 898–904. [Online]. Available: <https://doi.org/10.1145/3716368.3735294>
- [18] W. Maass, “Liquid computing,” in *Computation and Logic in the Real World*, S. B. Cooper, B. Löwe, and A. Sorbi, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 507–516.
- [19] S. Schlag, T. Heuer, L. Gottesbüren, Y. Akhremtsev, C. Schulz, and P. Sanders, “High-quality hypergraph partitioning,” *ACM J. Exp. Algorithmics*, vol. 27, Feb. 2023. [Online]. Available: <https://doi.org/10.1145/3529090>
- [20] C. Fernandez Musoles, “Improving scalability of large-scale distributed spiking neural network simulations on high performance computing systems using novel architecture-aware streaming hypergraph partitioning,” December 2020, unpublished. [Online]. Available: <https://theses.whiterose.ac.uk/id/eprint/29007/>
- [21] D. Zhou, J. Huang, and B. Schölkopf, “Learning with hypergraphs: Clustering, classification, and embedding,” in *Advances in Neural Information Processing Systems*, B. Schölkopf, J. Platt, and T. Hoffman, Eds., vol. 19. MIT Press, 2006. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2006/file/dff8e9c2ac33381546d96deea9922999-Paper.pdf
- [22] B. Rueckauer, I.-A. Lungu, Y. Hu, M. Pfeiffer, and S.-C. Liu, “Conversion of continuous-valued deep networks to efficient event-driven networks for image classification,” *Frontiers in neuroscience*, vol. 11, p. 682, 2017.
- [23] W. Fang, Y. Chen, J. Ding, Z. Yu, T. Masquelier, D. Chen, L. Huang, H. Zhou, G. Li, and Y. Tian, “Spikingjelly: An open-source machine learning infrastructure platform for spike-based intelligence,” *Science Advances*, vol. 9, no. 40, p. eadi1480, 2023. [Online]. Available: <https://www.science.org/doi/abs/10.1126/sciadv.adi1480>

- [24] L. Deng, Y. Wu, X. Hu, L. Liang, Y. Ding, G. Li, G. Zhao, P. Li, and Y. Xie, "Rethinking the performance comparison between snns and anns," *Neural Networks*, vol. 121, pp. 294–307, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0893608019302667>
- [25] M. Xiao, Q. Meng, Z. Zhang, Y. Wang, and Z. Lin, "Training feedback spiking neural networks by implicit differentiation on the equilibrium state," in *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 14 516–14 528. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2021/file/79a49b3e3762632813f9e35f4ba53d6c-Paper.pdf
- [26] Y. Liu, Y. Jin, and P. Li, "Online adaptation and energy minimization for hardware recurrent spiking neural networks," *J. Emerg. Technol. Comput. Syst.*, vol. 14, no. 1, Jan. 2018. [Online]. Available: <https://doi.org/10.1145/3145479>
- [27] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, "Line: Large-scale information network embedding," in *Proceedings of the 24th International Conference on World Wide Web*, ser. WWW '15. Republic and Canton of Geneva, CHE: International World Wide Web Conferences Steering Committee, 2015, p. 1067–1077. [Online]. Available: <https://doi.org/10.1145/2736277.2741093>
- [28] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. USA: W. H. Freeman & Co., 1990.
- [29] S. Schlag, V. Henne, T. Heuer, H. Meyerhenke, P. Sanders, and C. Schulz, "k-way hypergraph partitioning via n-level recursive bisection," 2015. [Online]. Available: <https://arxiv.org/abs/1511.03137>
- [30] G. Karypis and V. Kumar, "Multilevel k-way hypergraph partitioning," in *Proceedings of the 36th Annual ACM/IEEE Design Automation Conference*, ser. DAC '99. New York, NY, USA: Association for Computing Machinery, 1999, p. 343–348. [Online]. Available: <https://doi.org/10.1145/309847.309954>
- [31] A. B. Kahn, "Topological sorting of large networks," *Commun. ACM*, vol. 5, no. 11, p. 558–562, Nov. 1962. [Online]. Available: <https://doi.org/10.1145/368996.369025>
- [32] Y. Koren, "Drawing graphs by eigenvectors: theory and practice," *Computers & Mathematics with Applications*, vol. 49, no. 11–12, pp. 1867–1888, 2005.
- [33] M. Belkin and P. Niyogi, "Laplacian eigenmaps for dimensionality reduction and data representation," *Neural Computation*, vol. 15, no. 6, pp. 1373–1396, 2003.
- [34] R. B. Lehoucq, D. C. Sorensen, and C. Yang, *ARPACK Users' Guide*. Society for Industrial and Applied Mathematics, 1998. [Online]. Available: <https://epubs.siam.org/doi/abs/10.1137/1.9780898719628>
- [35] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2015. [Online]. Available: <https://arxiv.org/abs/1409.1556>
- [36] F. Chollet *et al.*, "Keras," <https://keras.io>, 2015.
- [37] A. Paszke *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019.
- [38] Y. N. Billeh *et al.*, "Systematic integration of structural and functional data into multi-scale models of mouse primary visual cortex," *Neuron*, vol. 106, no. 3, pp. 388–403.e18, May 2020. [Online]. Available: <https://doi.org/10.1016/j.neuron.2020.01.040>
- [39] A. Roxin, N. Brunel, D. Hansel, G. Mongillo, and C. van Vreeswijk, "On the distribution of firing rates in networks of cortical neurons," *J Neurosci*, vol. 31, no. 45, pp. 16 217–16 226, Nov. 2011.